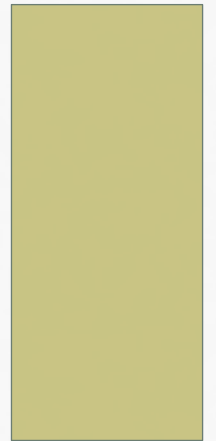


FORTRAN 90

SUBPROGRAMAS



FUNCIONES Y SUBROUTINAS

- Fortran 90 tiene dos tipos de subprogramas, **funciones** y *subrutinas*.
- Una **función** en Fortran 90 regresa un resultado calculado a través del nombre de la **función**.
- Si una **función** no tiene que regresar un valor, utiliza una *subrutina*.

SINTAXIS DE UNA FUNCIÓN

- Una función tiene la siguiente sintaxis:

```
type PROGRAM nombre-funcion (arg1, arg2, ..., argn)
  IMPLICIT NONE
  [Seccion de especificaciones]
  [Seccion de ejecucion]
  [Seccion de subprogramas]
END OF PROGRAM nombre-funcion
```

- **Type** es un tipo de Fortran 90, por ejemplo: **INTEGER**, **REAL**, **LOGICAL**, etc... con o sin **KIND**.
- **Nombre-funcion** es un identificador de Fortran 90.
- **arg1, arg2, ..., argn**, son argumentos formales.

SINTAXIS DE UNA FUNCIÓN

- Una función es una unidad autocontenida que recibe una “entrada” via sus argumentos formales, hace algunos cálculos, y regresa un resultado con el nombre de dicha función.
- En alguna parte de la función debe haber una o más declaraciones como ésta:

nombre-función = expresión

Donde el resultado de expresión se guarda en el nombre de la función.

Es importante hacer notar que el nombre de la función **nombre-función** no puede aparecer en el lado derecho de ninguna expresión.

SINTAXIS DE UNA FUNCIÓN

- Una función es una unidad autocontenida que recibe una “entrada” via sus argumentos formales, hace algunos cálculos, y regresa un resultado con el nombre de dicha función.
- En alguna parte de la función debe haber una o más declaraciones como ésta:

nombre-función = expresión

Donde el resultado de expresión se guarda en el nombre de la función.

Es importante hacer notar que el nombre de la función **nombre-función** no puede aparecer en el lado derecho de ninguna expresión.

- En la especificación de tipo **type**, los argumentos formales deben tener un nuevo atributo **INTENT(IN)**.
- El significado de **INTENT(IN)** es que la función solo toma su valor de un argumento formal y no cambia su contenido.
- Cualquier declaración que pueda usarse en **PROGRAM** puede usarse en **FUNCTION**.

EJEMPLO DE FUNCIONES

- Es importante hacer notar que las funciones pueden no tener un argumento formal, pero () es aún requerido.

```
!Obtencion de un Factorial
INTEGER FUNCTION Factorial(n)
  IMPLICIT NONE
  INTEGER, INTENT(IN) :: n
  INTEGER :: i, Ans

  Ans = 1
  DO i = 1, n
    Ans = Ans * i
  END DO
  Factorial = Ans
END FUNCTION Factorial
```

```
!Lee y regresa un numero real positivo
REAL FUNCTION ObtenNumero()
  IMPLICIT NONE
  REAL :: Valor_Entrada
  DO
    WRITE(*,*) 'Un numero positivo: '
    READ(*,*) Valor_Entrada
    IF (Valor_Entrada > 0.0) EXIT
    WRITE(*,*) 'ERROR. Intenta de nuevo.'
  END DO
  ObtenNumero = Valor_Entrada
END FUNCTION ObtenNumero
```

USO DE LAS FUNCIONES

- El uso de una función definida por el usuario, es similar al de una función intrínseca de Fortran 90.
- La siguiente sentencia utiliza `Factorial(n)` para calcular la combinatoria $C(m,n)$ donde `m` y `n` son argumentos:

```
Cmn = Factorial(m)/(Factorial(n)*Factorial(m-n))
```

- Hay que hacer notar que una combinatoria se define de la siguiente manera:

$$C(m, n) = \frac{m!}{n! \times (m - n)!}$$

¿EN DONDE COLOCAR LAS FUNCIONES?

- Las funciones en Fortran 90 pueden ser internas o externas.
- Las funciones internas se encuentran dentro de un PROGRAM, por ejemplo:

```
PROGRAM nombre-programa  
  IMPLICIT NONE  
  [seccion de especificaciones]  
  [sección de ejecuciones]
```

```
CONTAINS  
  [funciones]  
END PROGRAM nombre-programa
```

- Aunque una función puede contener otras funciones, las funciones internas no pueden contener otras funciones.

¿EN DONDE COLOCAR LAS FUNCIONES?

- El programa de la derecha muestra dos funciones internas, **PromArit()** y **PromGeom()**.
- Estas funciones toman dos argumentos **REAL**, calculan y regresan valor de función **REAL**.

```
PROGRAM DosFunciones
  IMPLICIT NONE
  REAL :: a, b, A_Prom, G_Prom
  READ(*,*) a, b
  A_Prom = PromArit(a, b)
  G_Prom = PromGeom(a,b)
  WRITE(*,*) a, b, A_Prom, G_Prom
CONTAINS
  REAL FUNCTION PromArit(a, b)
    IMPLICIT NONE
    REAL, INTENT(IN) :: a, b
    PromArit = (a+b)/2.0
  END FUNCTION PromArit
  REAL FUNCTION PromGeom(a, b)
    IMPLICIT NONE
    REAL, INTENT(IN) :: a, b
    PromGeom = SQRT(a*b)
  END FUNCTION PromGeom
END PROGRAM DosFunciones
```

EJEMPLO

- Si un triángulo tiene las longitudes de sus lados dadas por a , b y c , la fórmula de Herón calcula el área del triángulo de la siguiente manera:

$$Area = \sqrt{s \times (s - a) \times (s - b) \times (s - c)}$$

donde s es el semi-perímetro del triángulo:

$$s = \frac{a+b+c}{2}$$

- Para formar un triángulo las longitudes a , b , c deben de cumplir las siguientes condiciones:
 - $a > 0$, $b > 0$ y $c > 0$
 - $a + b > c$, $a + c > b$ y $b + c > a$

EJEMPLO

- La función **PruebaTriangulo()** de tipo **LOGICAL** se asegura de que todos los lados sean positivos, y que la suma de dos lados cualquiera sea mayor que el tercero.

```
LOGICAL FUNCTION PruebaTriangulo(a, b, c)
  IMPLICIT NONE
  REAL, INTENT(IN) :: a, b, c
  LOGICAL          :: test1, test2
  test1 = (a > 0.0) .AND. (b > 0.0) .AND. (c > 0.0)
  test2 = (a + b > c) .AND. (a + c > b) .AND. (b + c > a)
  PruebaTriangulo = test1 .AND. test2 ! Los dos deben ser .TRUE.
END FUNCTION PruebaTriangulo
```

EJEMPLO

- La siguiente función implementa la fórmula de Herón.

```
REAL FUNCTION Area(a, b, c)
  IMPLICIT NONE
  REAL, INTENT(IN) :: a, b, c
  REAL              :: s
  s = (a + b + c) / 2.0
  Area = SQRT(s*(s-a)*(s-b)*(s-c))
END FUNCTION Area
```

EJEMPLO

- Programa completo:

```
PROGRAM FormulaHeron
  IMPLICIT NONE
  REAL :: a, b, c, AreaTriangulo
  DO
    WRITE(*,*) 'Introduce la longitud de los tres lados de un &
      triangulo --> '
    READ(*,*) a, b, c
    WRITE(*,*) 'La longitud de los lados son: ', a, b, c
    IF (PruebaTriangulo(a, b, c)) EXIT ! Salir si forman un triangulo
    WRITE(*,*) 'Tus valores no pueden formar un triangulo. Intentalo de nuevo'
  END DO
  AreaTriangulo = Area(a, b, c)
  WRITE(*,*) 'El Area del Triangulo es ', AreaTriangulo

CONTAINS
LOGICAL FUNCTION PruebaTriangulo(a, b, c)
  .....
END FUNCTION PruebaTriangulo
REAL FUNCTION Area(a, b, c)
  .....
END FUNCTION Area
END PROGRAM FormulaHeron
```

SUBROUTINAS

- Una función de Fortran 90 toma valores de sus argumentos formales, y devuelve un solo valor con el nombre de la función.
- Una subrutina de Fortran 90 toma valores de sus argumentos formales y regresa algunos resultados calculados con sus argumentos formales.
- Una subrutina de Fortran 90 no regresa ningun valor con su nombre.
- La siguiente es la sintaxis de una subrutina en Fortran 90:

```
SUBROUTINE nombre-subrutina(arg1,arg2,...,argn)
  IMPLICIT NONE
  [Seccion de especificaciones]
  [Seccion de ejecucion]
  [Seccion de subprograma]
END SUBROUTINE nombre-subrutina
```

EL ATRIBUTO **INTENT()**

- Dado que las subrutinas utilizan argumentos formales para recibir valores y regresar resultados, en adición a **INTEN(IN)**, existen **INTEN(OUT)** e **INTEN(INOUT)**.
- **INTEN(OUT)** significa que un argumento formal no recibe un valor; pero, regresará un valor a su argumento correspondiente.
- **INTEN(INOUT)** significa que un argumento formal recibe un valor y regresa otro a su argumento correspondiente.

EL ATRIBUTO INTENT()

- Por ejemplo:

! Am, Gm y Hm se usan para regresar los resultados

```
SUBROUTINE Promedios(a, b, c, Am, Gm, Hm)
```

```
  IMPLICIT NONE
```

```
  REAL, INTENT(IN) :: a, b, c
```

```
  REAL, INTENT(OUT) :: Am, Gm, Hm
```

```
  Am = (a+b+c)/3.0
```

```
  Gm = (a*b*c)**(1.0/3.0)
```

```
  Hm = 3.0/(1.0/a + 1.0/b + 1.0/c)
```

```
END SUBROUTINE Promedios
```

! Los valores a y b son intercambiados

```
SUBROUTINE Swap(a, b)
```

```
  IMPLICIT NONE
```

```
  INTEGER, INTENT(INOUT) :: a, b
```

```
  INTEGER :: c
```

```
  c = a
```

```
  a = b
```

```
  b = c
```

```
END SUBROUTINE Swap
```


LA DECLARACIÓN CALL

- A diferencia de C/C++ y Java, Fortran 90 utiliza la sentencia CALL, para utilizar una subrutina.
- La sentencia CALL puede tener una de las siguientes formas:
 - CALL sub-nombre(arg1,arg2,...,argn)
 - CALL sub-nombre()
 - CALL sub-nombre
- Las últimas dos son equivalentes y se utilizan para llamar a una subrutina sin argumentos formales.

```
PROGRAM Prueba
  IMPLICIT NONE
  REAL :: a, b
  READ(*,*) a, b
  CALL Swap(a,b)
  WRITE(*,*) a, b
CONTAINS
  SUBROUTINE Swap(x,y)
    IMPLICIT NONE
    REAL, INTENT(INOUT) :: x,y
    REAL :: z
    z = x
    x = y
    y = z
  END SUBROUTINE Swap
END PROGRAM Prueba
```